RRSS
Coding Standards

1 Introduction

This document entails the coding standards that'll be used in our project. This document is comprised of two sections: description, in which we'll explain why coding standards are necessary and what kind of standards are outlined in this document and the specification, in which we'll be outlining the actual specification that'll be used during the development of this project.

2 Description

Coding standards are necessary in order to ease the cost of maintenance on the developers of software and help ensure a certain degree of quality for the code. The standards which will be outlined in the further sections of this document will be pertaining to both the semantic style(what kind of structures to avoid, what kind of code to write etc.) and the visual style(how many spaces to use etc.). Specifically this document contains the following sections: naming standards, file organization, comment standards and coding conventions. Please note that this document is not final, the standard will continue to evolve as the project is developed according to the needs.

3 Coding Standards Specifications

3.1 Naming Standard

- Constants and enum values will be in upper snake case. (FOO_BAR)
- HTML name and ID attributes, class names and CSS variable names will be in kebab case. (foo-bar)
- Java class names will be in pascal case. (FooBar)
- Java variable and method names will be in camel case. (fooBar)

3.2 File Organization

- Thymeleaf HTML templates will be stored in src/main/resources/templates/.
- Static files will be stored in src/main/resources/static/.
- Data Transfer Object code will be stored in src/main/java/com/fosskeeters/rrss/dtos/.
- Controller code will be stored in src/main/java/com/fosskeeters/rrss/controllers/.
- Models will be stored in src/main/java/com/fosskeeters/rrss/models/.
- Unit tests will be stored in src/test/java/com/fosskeeters/rrss/.
- Functional tests will be stored in test/.
- Build related files will be stored in the root of the repository.
- Spring boot property files will be stored in src/main/resources/*.properties.
- CI workflows will be stored in .github/workflows.
- Documentation will be stored in docs/.

3.3 Comment Standards

Since our aim is to have clean, readable code, for most cases we will be commenting not what a piece of code does but why the code is necessary.

3.4 Coding Conventions

Most of the conventions below are enforced using the clang-format linter. See src/main/java/.clang-format and .github/workflows/ci.yml for more information.

- All code will use 4 spaces indentation.
- Java: Braces for a block will be placed on the same line as the block itself.
- Java: No line will exceed 100 characters.
- Java: Function parameters/arguments on a different line will be aligned to the arguments in the previous line.

- Java: In binary operations between variables and literals, variables will be placed in the left-side of the operator. (a.k.a no yoda statements.)
- Java: Any significant number will be made a constant. (a.k.a. no magic numbers.)
- Java: Assertions will be used to denote assumptions.
- Java: Double negatives will be avoided as much as possible. (ex. !noDisableFooBar)
- Java: Boolean literals passed to functions will be prefixed by a comment for the argument name. (ex. Method.foo(/*bar=*/true))
- CSS: Every subsection of CSS will be separated by a comment header.
- CSS: Significant values will be made variables.